

GROUP PROJECT: STRING COMPRESSION

{lftung,thquan}@fit.hcmus.edu.vn

CSC10004 - Summer 2026

Contents

1	Project Description	1
2	Research	1
2.1	Coding	1
2.2	Experiments	2
2.3	Bonus Points: Advanced Algorithms	2
3	Report	3
3.1	Group Information	3
3.2	Problem Statement	3
3.3	Algorithm Analysis	3
3.4	Experimental Results	3
3.5	References	4
3.6	Acknowledgment	4
4	Submission & Scoring Rubrics	4

1 Project Description

For this project, your main task is to investigate the problem of *data compression*, specifically focusing on lossless compression techniques. While it might seem like a behind-the-scenes utility, data compression is actually a fundamental concept in computer science with massive real-world implications. It allows us to efficiently manage limited storage, transmit data rapidly over networks, and forms the backbone of ubiquitous technologies like ZIP file archivers and PNG image formats. To get started, you should research and compare the following core algorithms:

- Run-Length Encoding (RLE).
- Huffman Coding.
- The Lempel-Ziv-Welch (LZW) algorithm.

For a solid theoretical foundation, we highly recommend reading **Chapter 16: Greedy Algorithms** (specifically the section on Huffman codes) in *Introduction to Algorithms* (3rd Edition) by Thomas H. Cormen. It covers the mechanism and mathematical justification for building optimal prefix codes in detail. If you don't have the physical textbook handy, you can reference the [PDF version here](#).

2 Research

2.1 Coding

For the coding part, you will develop a command-line utility to read an input text file, compress or decompress it using a specified algorithm, and output the result.

```
Usage: compressor[.exe] -a [algorithm] -m [mode] -i [input_file] -o [
    output_file]

Options:
  -a [algorithm]  Select algorithm: rle, huff, lzw
  -m [mode]       Select mode: c (compress), d (decompress)
  -i [input_file] Path to the source file
  -o [output_file] Path to the resulting file
```

Important We will compile your code using the GNU g++ compiler with the C++17 standard. The exact command used will be:

```
> g++ file1.cpp [...] -o compressor[.exe] -std=c++17
```

Any submission containing non-standard libraries will fail to compile and may receive a grade of 0.

Input and Output

Input The input structure depends on the selected mode:

- **Compression Mode** (-m c): A standard text file containing standard ASCII characters.
- **Decompression Mode** (-m d): A previously compressed file generated by your tool. Note that depending on the algorithm (like Huffman or LZW), this file might contain binary data rather than human-readable text.

Output

- **File Output:** The tool must generate the resulting compressed or decompressed file and save it to the specified -o path.
- **Console Output:** Print the performance summary to the standard output (console), separated by a dashed line.

Notes on performance metrics

At the end of every compression execution, students must output a performance summary containing the following metrics:

- **Execution Time:** Measured in milliseconds (ms) for the core algorithm.
- **Compression Ratio:** Calculated as shown in Equation (1):

$$\text{Compression Ratio} = \frac{\text{Uncompressed Size (in bytes)}}{\text{Compressed Size (in bytes)}} \quad (1)$$

- **Space Savings:** Calculated as shown in Equation (2):

$$\text{Space Savings} = 1 - \frac{\text{Compressed Size (in bytes)}}{\text{Uncompressed Size (in bytes)}} \quad (2)$$

Example

Below is an example of the command execution and the expected console output when compressing a highly repetitive text file using Run-Length Encoding (RLE).

Input File (puzzle.txt)	Console Output
AAAAABBBBBCCCCDDDDD	Compression complete. ----- Algorithm: RLE Execution Time: 0.05 ms Original Size: 20 bytes Compressed Size: 8 bytes Compression Ratio: 2.5 Space Savings: 60.0%

Note: In this example, the uncompressed text is 20 characters (20 bytes). The RLE algorithm compresses it to 8 bytes (e.g., 5A5B5C5D), resulting in a 60% space savings.

2.2 Experiments

Students are required to benchmark their algorithms to verify efficiency and scaling. You must vary the input parameters to generate meaningful analytical data.

- **Scenario 1 (Impact of File Size):** Fix the data type (e.g., standard English text). Run tests with increasing file sizes ranging from small (10 KB) to large (10 MB).
- **Scenario 2 (Impact of Data Entropy):** Fix the file size to a medium value (e.g., 1 MB). Run tests on different types of data: highly repetitive text (e.g., repeating "AAAAABBBBB"), standard English prose, and completely randomized character strings.

2.3 Bonus Points: Advanced Algorithms

Students can earn up to 1 bonus point by researching, implementing, and analyzing an advanced algorithm, such as Arithmetic Coding or Deflate (a combination of LZ77 and Huffman coding). This requires full integration into the command-line tool.

3 Report

You will write a report (in **English**) following this outline:

3.1 Group Information

Please list your group members and their specific contributions in the table below:

Student Name	Student ID	Assigned Tasks	% Completed
Name 1	ID 1	e.g., Huffman Research, Report Drafting	100%
Name 2	ID 2	e.g., LZW Code, Proofreading	100%
Name 3	ID 3	...	...
Total Group Completion			100%

3.2 Problem Statement

Provide a formal definition of the lossless data compression problem. You must also define a specific running example (e.g., a short, repetitive string like **BABBACAC**) to demonstrate each algorithm below.

3.3 Algorithm Analysis

For each algorithm (Run-Length Encoding, Huffman Coding, and LZW), your report must include the following five sections:

- **Introduction & Application:** Briefly explain the core idea behind the algorithm. Mention where it is most commonly used in the real world (e.g., RLE in fax machines, Huffman in JPEG, LZW in GIF).
- **Pseudocode:** Provide the pseudocode for both the compression and decompression phases. **Do not copy-paste or screenshot your source code here.** Write abstract, language-agnostic pseudocode.
- **Step-by-step trace:** Using your defined running example, demonstrate the internal state changes at each step.
 - For **RLE**, show the count accumulating.
 - For **Huffman**, show the step-by-step construction of the min-heap and the resulting binary tree.
 - For **LZW**, provide a table showing the dictionary expanding dynamically as new substrings are read.
- **Complexity:** Specify the formal mathematical time and space complexity using Big-O notation. Account for parameters such as the length of the input text (N) and the size of the alphabet/dictionary (K).
- **Discussion:** Critically analyze the algorithm's performance boundaries. Discuss the best-case and worst-case scenarios. When does RLE cause negative compression? When does LZW's initial dictionary overhead hurt performance?

3.4 Experimental Results

In this section, you are required to benchmark your algorithms to verify their efficiency across the defined scenarios (Impact of File Size and Impact of Data Entropy). You must present your measurements, followed by a discussion of the results.

You are required to report the performance metrics in a table format (see Table 1). While visualization (charts/graphs) is optional, it is highly recommended to help illustrate trends.

Table 1: Example format for reporting execution time and compression ratio measurements.

File Size	Data Type	RLE	Huffman	LZW
10 KB	Standard English	0.10 ms (0.9x)	0.45 ms (1.8x)	0.60 ms (2.1x)
1 MB	Standard English	12.50 ms (0.9x)	45.00 ms (1.8x)	62.00 ms (2.3x)
1 MB	Highly Repetitive	2.10 ms (15.0x)	15.00 ms (4.0x)	25.00 ms (10.0x)

Based on the data above, provide your comments on the performance of the algorithms:

- Does the growth rate in your data match the Big-O complexity you analyzed in Section 3.3?
- Which algorithm achieved the highest compression ratio on standard text versus highly repetitive data?
- Explain why you think these results happened based on the theoretical limits of each algorithm.

3.5 References

You must cite all resources used in your research (textbooks, papers, websites) following the [IEEE Citation Style](#).

3.6 Acknowledgment

Support was provided by Mr./Ms./[Relationship] ABCD in developing the code for the XYZ function, among others. In addition, AI tools, such as ChatGPT/Gemini/DeepSeek, assisted with drafting Section I of the report and implementing the XYZ function.

4 Submission & Scoring Rubrics

You will work in a group of 3-5 students. Students must prepare all the following content, compress it into a <GroupID>.zip file (replace <GroupID> with the group ID number) and submit it on Moodle before the deadline:

1. **Report:** The report file `Report.pdf`.
2. **Demonstration Video:** Submit a video (max 5 mins) verifying successful compilation and functionality. Upload it to YouTube (**Unlisted/Public**) with the title
25CA-DSA-Group<GroupID> String Compression
Submit the link in a file named `video.txt`.
3. **Source Code:** Submit a `source` folder containing your code, a `readme.txt` guide, and a `tests` subfolder with your test cases. If the file size exceeds Moodle's limit, include a download link in the `readme.txt`.

We will use the following grading rubric:

Important: Criteria marked in **red** are **mandatory**. If any mandatory item is missing, you will receive a score of **0** for that **entire category** (Report, Video, or Source Code). Submissions that fail to follow the submission instructions will automatically receive a grade of 0 for the project.

Category	Criteria	Weight
Report (5 pts)	Researching content & presenting algorithms (must follow Section 3)	60%
	Experimental results	20%
	References	10%
	Report format & content presentation	10%
Demo Video (2 pts)	Following the video submission rules	40%
	Presenting the compilation phase and built algorithms completely and concisely	50%
	Having timestamps to mark each step	10%
Source Code (3 pts)	Source code (C/C++ only) compiles and runs as shown in demo	50%
	Having detailed installation and user guides	30%
	Well-organized source code	20%